

실습환경 구축-3

01 SpringBoot 프레임워크를 활용한 웹프로젝트 구축(2)

학습 목표

- ✓ SpringBoot 프로젝트 만들기
- ✓ SpringBoot 기반의 게시판 프로그램 작성



학습 내용

1. 게시판 코드의 구조 이해
2. 코드 입력과 빌드, 실행



실습환경 구축-3

01 SpringBoot 프레임워크를 활용한 웹프로젝트 구축(2)

Spring Boot을 이용한 간단한 게시판 만들기

Step 1. 설계 (게시판 기능 정의 및 아키텍처)

✓ 주요 기능

- 게시글 작성 (Create)
- 게시글 목록 조회 (Read)
- 게시글 상세 조회 (Read)
- 게시글 수정 (Update)
- 게시글 삭제 (Delete)

✓ 데이터 모델(Entity)

필드명	데이터 타입	설명
id	Long(PK)	게시글 ID
title	String	제목
content	String	내용
createAt	LocalDateTime	생성 날짜

Spring Boot을 이용한 간단한 게시판 만들기

Step 1. 설계 (게시판 기능 정의 및 아키텍처)

✓ API 설계

기능	HTTP Method	URL	요청 데이터	응답 데이터
게시글 작성	POST	/posts	{title, content}	{id, title, content, createAt}
모든 게시글 조회	GET	/posts	-	{id, title, content, createAt}
게시글 상세 조회	GET	/posts/{id}	-	{id, title, content, createAt}
게시글 수정	PUT	/posts/{id}	{title, content}	{id, title, content, createAt}
게시글 삭제	DELETE	/posts/{id}	-	성공/실패 메시지

Spring Boot을 이용한 간단한 게시판 만들기

Step 2. 환경설정

1. Spring Boot 프로젝트 생성

방법 1: Spring Initializr 사용(<https://start.spring.io/>)

방법 2: STS (Spring Tool Suite)에서 New -> Spring Starter Project

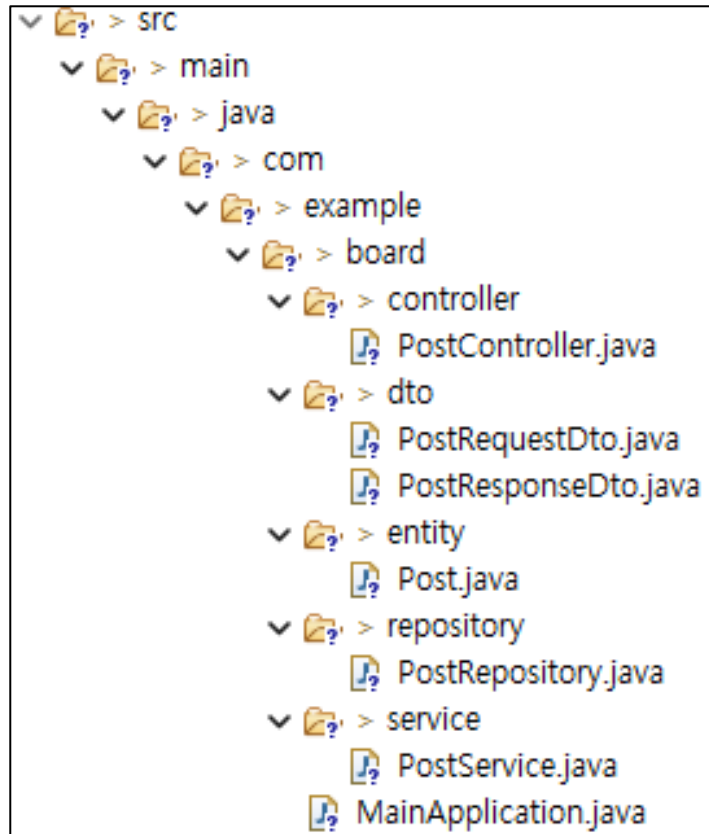
2. 프로젝트 설정

- Project Type: Maven
- Language : Java
- Spring Boot Version : 최신 안정 버전
- Dependencies(필수 라이브러리) 추가
 - Spring Web(Rest API 개발)
 - Spring Data JPA(DB연동)
 - H2 Database(테스트용 인메모리 DB)
 - Lombok(코드 간소화)

Spring Boot을 이용한 간단한 게시판 만들기

Step 3. 코드 개발

1. 프로젝트 구조



패키지 종류별 역할

- controller: 사용자 요청 받아들이는 역할
- service: 비즈니스 로직을 수행하는 역할
- repository: DB와 직접 통신하는 역할
- dto: Data Transfer Object – 요청과 응답 데이터를 담는 클래스
- entity: DB 테이블과 연결되는 클래스

Spring Boot을 이용한 간단한 게시판 만들기

Step 3. 코드 개발

2. Entity (Post.java)

```
package com.example.board.entity;

import jakarta.persistence.*;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.LocalDateTime;

@Entity
@Getter
@Setter
@NoArgsConstructor
public class Post {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String content;
    private LocalDateTime createdAt = LocalDateTime.now();
}
```

@Entity → DB 테이블과 연결되는 객체
@Id, @GeneratedValue → 기본 키 자동 증가
LocalDateTime createdAt → 생성 날짜 저장

Spring Boot을 이용한 간단한 게시판 만들기

Step 3. 코드 개발

3. Repository (PostRepository.java)

```
package com.example.board.repository;

import com.example.board.entity.Post;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PostRepository extends JpaRepository<Post, Long> {
}
```

📌 설명

- JpaRepository<Post, Long> -> 기본 CRUD 메서드 자동 제공 (save(), findAll(), findById(), deleteById())

Spring Boot을 이용한 간단한 게시판 만들기

Step 3. 코드 개발

4. DTO (PostRequestDto.java, PostResponseDto.java)

✦ PostRequestDto.java (클라이언트에서 요청받는 데이터)

```
package com.example.board.dto;

import lombok.Getter;

@Getter
public class PostRequestDto {
    private String title;
    private String content;
}
```

Spring Boot을 이용한 간단한 게시판 만들기

Step 3. 코드 개발

✦ PostResponseDto.java (응답할 데이터)

```
package com.example.board.dto;

import com.example.board.entity.Post;
import lombok.Getter;

import java.time.LocalDateTime;

@Getter
public class PostResponseDto {
    private Long id;
    private String title;
    private String content;
    private LocalDateTime createdAt;

    public PostResponseDto(Post post) {
        this.id = post.getId();
        this.title = post.getTitle();
        this.content = post.getContent();
        this.createdAt = post.getCreatedAt();
    }
}
```

Spring Boot을 이용한 간단한 게시판 만들기

Step 3. 코드 개발

5. Service (PostService.java)

```
package com.example.board.service;

import com.example.board.dto.PostRequestDto;
import com.example.board.dto.PostResponseDto;
import com.example.board.entity.Post;
import com.example.board.repository.PostRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class PostService {
    private final PostRepository postRepository;

    public PostService(PostRepository postRepository) {
        this.postRepository = postRepository;
    }

    public List<PostResponseDto> getAllPosts() {
        return postRepository.findAll().stream()
            .map(PostResponseDto::new)
            .collect(Collectors.toList());
    }
}
```

```
    public PostResponseDto getPostById(Long id) {
        Post post = postRepository.findById(id).orElseThrow();
        return new PostResponseDto(post);
    }

    public PostResponseDto createPost(PostRequestDto requestDto) {
        Post post = new Post();
        post.setTitle(requestDto.getTitle());
        post.setContent(requestDto.getContent());
        postRepository.save(post);
        return new PostResponseDto(post);
    }

    @Transactional
    public PostResponseDto updatePost(Long id, PostRequestDto requestDto) {
        Post post = postRepository.findById(id).orElseThrow();
        post.setTitle(requestDto.getTitle());
        post.setContent(requestDto.getContent());
        return new PostResponseDto(post);
    }

    public void deletePost(Long id) {
        postRepository.deleteById(id);
    }
}
```

Spring Boot을 이용한 간단한 게시판 만들기

Step 3. 코드 개발

6. Controller (PostController.java)

```
package com.example.board.controller;

import com.example.board.dto.PostRequestDto;
import com.example.board.dto.PostResponseDto;
import com.example.board.service.PostService;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/posts")
public class PostController {
    private final PostService postService;

    public PostController(PostService postService) {
        this.postService = postService;
    }

    @GetMapping
    public List<PostResponseDto> getAllPosts() {
        return postService.getAllPosts();
    }
}
```

```
@GetMapping("/{id}")
public PostResponseDto getPost(@PathVariable Long id) {
    return postService.getPostById(id);
}

@PostMapping
public PostResponseDto createPost(@RequestBody PostRequestDto requestDto) {
    return postService.createPost(requestDto);
}

@PutMapping("/{id}")
public PostResponseDto updatePost(@PathVariable Long id, @RequestBody PostRequestDto requestDto) {
    return postService.updatePost(id, requestDto);
}

@DeleteMapping("/{id}")
public void deletePost(@PathVariable Long id) {
    postService.deletePost(id);
}
}
```

Spring Boot을 이용한 간단한 게시판 만들기

Step 4. 실행 및 테스트

1. Spring Boot 실행

- MainApplication.java 실행
- `http://localhost:8080/posts` 에서 API 테스트

2. Postman으로 CRUD 테스트

- `POST /posts` -> 게시글 생성
- `GET /posts` -> 모든 게시글 조회 (목록)
- `GET /posts/{id}` -> 특정 게시글 조회
- `PUT /posts/{id}` -> 게시글 수정
- `DELETE /posts/{id}` -> 게시글 삭제

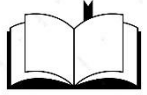
시연 영상

Spring Boot 게시판 만들기



요약 정리

- 게시판 API 구조를 CRUD 기능 중심으로 설계
- DB 테이블과 매핑하기 위해 Entity에 JPA Annotation을 반영
- Repository의 기본 기능은 JpaRepository를 상속하여 자동 구현
- 서비스 계층에서 DTO를 사용해 비즈니스 로직 처리
- Postman을 활용하여 API 기능을 직접 테스트 가능



참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [이지스퍼블리싱] Do it! 점프 투 스프링부트 3(2023.12)
- SpringBoot - <https://spring.io/>
- Spring Initializ - <https://start.spring.io/>
- Postman - <https://www.postman.com/>

The background is a vibrant blue gradient. It is decorated with various abstract elements: a large solid blue sphere in the upper left, a smaller sphere with diagonal stripes in the upper right, and another striped sphere in the lower left. Several network diagrams, consisting of white dots connected by thin white lines, are scattered across the image. There are also solid blue rectangular bars of different sizes and orientations. The overall aesthetic is modern and technological.

감사합니다.



퀴즈

1. 다음 중, Spring Boot 개발 방법론에서 계층별 설명이 잘못된 것은?

- | | |
|-----------------------------------|--------------------------|
| 1) Controller - Request에 대한 처리 로직 | 2) DTO - 데이터 엔티티 모델링 |
| 3) Repository - 데이터베이스와 상호작용 담당 | 4) Service - 화면 UI 처리 로직 |

2. 게시판 만들기 실습에서 사용한 테스트용 In-Memory DB의 이름은? (2글자)

()

3. 다음 중, 게시판 만들기 실습 내용에 대한 설명 중 잘못된 것은?

- | | |
|--------------------------------|--------------------------------|
| 1) PUT /posts - 게시글 생성 | 2) DELETE /posts/{id} - 게시글 삭제 |
| 3) GET /posts/{id} - 특정 게시물 조회 | 4) GET /posts - 게시글 목록 조회 |



퀴즈 정답

1. 다음 중, Spring Boot 개발 방법론에서 계층별 설명이 잘못된 것은?

4) Service - 화면 UI 처리 로직

서비스 계층은 비즈니스 로직을 처리하는 계층입니다.

2. 게시판 만들기 실습에서 사용한 테스트용 In-Memory DB의 이름은? (2글자)

(H2)

3. 다음 중, 게시판 만들기 실습 내용에 대한 설명 중 잘못된 것은?

1) PUT /posts - 게시글 생성

게시글을 생성할 때에는 POST를 사용하며, PUT은 게시글을 수정할 때 사용합니다.